

FICHE DE REVISION COMPLETE - IoT

Chapitres 1 a 5 + Partie code ESP32/Arduino + questions types examen

Objectif : reviser tout le cours de maniere structuree, sans repeter inutilement, en reliant les chapitres entre eux :
capter -> traiter -> transmettre -> stocker -> analyser -> agir.

Base de revision : cours du professeur, exercices de preparation, examens blancs et messages de revision recus. Le document est concu pour repondre aux questions de cours, questions a completer, lecture de code et exercices de raisonnement.

Plan logique du module

Chapitre	Question centrale	Lien avec les autres chapitres
Chapitre 1 - Introduction	C est quoi l IoT et comment est structuree une architecture IoT ?	Donne la vision globale : objets, reseau, application, metier.
Chapitre 2 - Acquisition	Comment transformer une grandeur physique en donnee numerique ?	Capteurs/actionneurs/microcontroleur produisent les donnees.
Chapitre 3 - Communication	Comment transmettre les donnees ?	MQTT, CoAP, HTTP, Wi-Fi, BLE, Zigbee, LoRaWAN, NB-IoT.
Chapitre 4 - Pretraitement	Comment rendre les donnees propres et utiles avant stockage ?	Filtrage, outliers, moyenne glissante, Edge/Fog/Cloud, bases.
Chapitre 5 - Integration	Comment construire un pipeline complet vers bases/cloud/dashboard ?	ETL, NiFi, Kafka, InfluxDB, MongoDB, PostgreSQL, cloud.
Partie code	Comment lire/expliquer/completer un code IoT ?	setup/loop, delay/millis, INPUT_PULLUP, capteur, relais, cloud.

0. Schema mental global a memoriser

L IoT relie le monde physique au monde numerique. La chaine complete est :

Grandeur physique -> Capteur -> Signal electrique -> Microcontrôleur -> Pretraitement Edge -> Protocole -> Broker/API -> ETL -> Base de donnees -> Dashboard/IA -> Action

Exemple fil conducteur : un lampadaire intelligent mesure la luminosite avec une LDR, detecte une presence avec un PIR, decide localement avec un ESP32, communique par MQTT/LoRaWAN, stocke dans InfluxDB, affiche dans Grafana et declenche une maintenance si une panne est detectee.

Chapitre 1 - Introduction a l'Internet des Objets

1. Definition de l'IIoT

L Internet des Objets (IIoT - Internet of Things) designe l'interconnexion de dispositifs physiques via Internet. Ces objets integrent des capteurs, des actionneurs, des processeurs/microcontrôleurs et des modules de communication afin de collecter, traiter, transmettre des donnees et parfois agir sur l'environnement sans intervention humaine directe.

Reponse type examen : L IIoT est l'interconnexion d'objets physiques via Internet permettant l'echange de donnees sans intervention humaine directe. Un objet IIoT contient des capteurs/actionneurs, un processeur, un module de communication et une source d'energie.

2. Concepts fondamentaux

Concept	Role	Exemple
Collecte de donnees	Mesurer l'environnement ou l'etat d'un objet.	DHT11 mesure temperature/humidite.
Connectivite	Transmettre les donnees entre objet, passerelle, cloud et application.	Wi-Fi, MQTT, LoRaWAN.
Traitement et analyse	Transformer les donnees brutes en information utile.	Calculer moyenne, detecter surchauffe.
Actionnement	Agir physiquement sur l'environnement.	Relais allume ventilateur ou lampe.

Phrase a retenir : un objet IIoT capte, communique, traite, puis agit.

3. Composants d'un objet connecte

Composant	Role	Exemples
Capteur	Mesure une grandeur physique.	LDR, DHT11, DS18B20, PIR, HC-SR04, MQ-2.
Actionneur	Recoit une commande et effectue une action.	LED, relais, moteur, buzzer, electrovanne.
Processeur / MCU	Lit les capteurs, applique la logique, controle les actionneurs.	ESP32, Arduino, Raspberry Pi.
Module de communication	Permet l'echange de donnees.	Wi-Fi, Bluetooth, LoRaWAN, Zigbee, NB-IoT.
Energie	Alimente le systeme, contrainte essentielle.	Batterie, secteur, panneau solaire.

4. Objet connecte vs ordinateur traditionnel

Un ordinateur traditionnel est polyvalent : il peut executer beaucoup de programmes mais consomme plus et n'est pas optimise pour une tache precise. Un objet IIoT est specialise : son hardware et son software sont concus pour une fonction specifique, donc il est plus efficace, moins couteux et consomme moins.

Reponse type : Un microcontrôleur ou un SBC est plus efficace qu'un ordinateur traditionnel pour une tache specifique car il est optimise pour cette tache avec moins de ressources et moins de consommation energetique.

5. M2M vs IoT

Notion	Définition	Limite / Extension
M2M	Machine to Machine : communication directe entre machines.	Souvent centré sur l'échange automatisé entre équipements.
IoT	Ecosystème plus large incluant objets, cloud, analyse, application et métier.	Ajoute dashboards, IA, stockage, intégration business.

6. Architectures IoT : 3, 5 et 7 couches

Architecture à 3 couches

Couche	Rôle	Contient
Perception	Collecter les données et agir sur le monde physique.	Capteurs, actionneurs, MCU, circuits.
Reseau	Transmettre les données vers passerelle, serveur ou cloud.	Wi-Fi, Bluetooth, Zigbee, LoRaWAN, MQTT, passerelle.
Application	Exploiter les données pour l'utilisateur.	Dashboard, application mobile, alertes, supervision.

Architecture à 5 couches

Couche	Rôle	Exemple
Perception	Mesure/action terrain.	LDR, DHT22, PIR, relais.
Transport / Reseau	Transmission des données.	Wi-Fi, MQTT, LoRaWAN.
Traitement	Filtrage, stockage préliminaire, analyse.	Edge, NiFi, Kafka, base.
Application	Service et interface utilisateur.	Grafana, app web/mobile.
Métier	Décision business et règles métier.	Maintenance, économie d'énergie, sécurité.

Architecture à 7 couches

Couche	Rôle
1. Objets / perception	Capteurs et actionneurs.
2. Connectivité	Communication locale ou longue portée.
3. Edge / passerelle	Prétraitement local et aggrégation.
4. Ingestion	Réception des flux MQTT/HTTP/Kafka.
5. Stockage	BDD relationnelle, NoSQL, time-series, data lake.
6. Analyse / traitement	Règles, IA, détection anomalies.
7. Application / métier	Dashboard, décision, supervision, contrôle.

7. Edge, Fog et Cloud - vision générale

Niveau	Lieu	Latence	Exemple
Edge	Objet, ESP32, capteur ou passerelle très proche.	Très faible.	ESP32 calcule une moyenne avant envoi.
Fog	Passerelle, routeur, serveur local intermédiaire.	Faible.	Gateway de quartier agrège les lampadaires.
Cloud	Datacenter distant.	Plus élevée.	AWS/Azure stocke et analyse historique.

Chapitre 2 - Acquisition des donnees IoT

1. Role de l'acquisition

L'acquisition transforme une grandeur physique en donnée numérique exploitable. C'est la première étape de toute solution IoT.

Monde physique -> Capteur -> Signal électrique -> Numérisation -> ESP32/Raspberry Pi -> Donnée exploitable

2. Capteur, actionneur et transducteur

Element	Definition	Exemples
Capteur	Mesure une grandeur physique et la convertit en signal exploitable.	DHT11, LM35, LDR, PIR, MQ-2.
Actionneur	Reçoit une commande et agit physiquement.	LED, relais, moteur, buzzer.
Transducteur	Partie du capteur qui transforme une grandeur physique en signal électrique.	LDR : lumière -> résistance ; thermocouple : température -> tension.

Phrase à retenir : le capteur observe, l'actionneur agit, le transducteur traduit le physique en électrique.

3. Capteurs analogiques, numériques et logiques TOR

Type	Signal	Lecture typique	Exemple
Analogique	Signal continu proportionnel à la grandeur.	analogRead()	LDR, LM35, potentiomètre.
Numérique	Valeur discrète codée sur n bits ou via protocole.	Bibliothèque / bus	DHT11, DHT22, DS18B20.
Logique TOR	Deux états seulement : 0/1, HIGH/LOW.	digitalRead()	PIR, bouton, fin de course.

4. Principes de transduction

Principe	Explication	Exemple
Resistif	Variation de résistance selon la grandeur.	LDR, thermistance, potentiomètre.
Capacitif	Variation de capacité.	Humidité, tactile.
Inductif	Variation d'inductance causée par métal/mouvement.	Détecteur de métaux.
Piezoelectrique	Charge électrique sous déformation mécanique.	Pression, force, vibration.
Optique	Détection/modulation de lumière ou infrarouge.	PIR, photodiode, barrière IR.

5. Capteurs importants à connaître

Capteur	Rôle	Détails examen
LDR	Mesure la luminosité.	Analogique ; utile pour jour/nuit et éclairage intelligent.
DHT11	Température + humidité.	Moins précis, faible coût, plage limitée.
DHT22	Température + humidité.	Plus précis que DHT11, plage plus large.
DS18B20	Température numérique.	Protocole 1-Wire, plage environ -55 deg C à +125 deg C.
PIR	Détection mouvement/présence.	Détecte variations infrarouges des corps

		en mouvement.
HC-SR04	Distance ultrasonique.	Calcule temps de vol de l onde sonore ; distance = $v \cdot t / 2$.
MQ-2	Detection gaz/fumee.	Gaz inflammables, securite IoT.

6. Thermistances et thermocouples

Thermistances

Type	Quand la temperature augmente	Usage
NTC	La resistance diminue.	Mesure de temperature.
PTC	La resistance augmente.	Protection, detection surchauffe.

Thermocouple et effet Seebeck

Un thermocouple est compose de deux metaux differents. Une difference de temperature entre les jonctions produit une tension : c est l effet Seebeck.

Formule : $V = S \times \Delta T$, ou V est la tension, S le coefficient de Seebeck et ΔT la difference de temperature.

Type	Composition	Plage / usage principal
K	Chromel - Alumel	-200 deg C a +1350 deg C ; polyvalent, industrie, fours.
J	Fer - Constantan	-40 deg C a +750 deg C ; industrie legere, attention corrosion.
T	Cuivre - Constantan	-200 deg C a +350 deg C ; basses temperatures, medical, cryogenie.

7. Microcontrôleur, ESP32 et SBC

Element	Definition	Usage
Microcontrôleur	Petit circuit programmable pour lire capteurs et commander actionneurs.	Arduino, ESP32.
ESP32	Microcontrôleur avec Wi-Fi/Bluetooth et GPIO.	Tres utilise pour prototypes IoT.
SBC	Single Board Computer avec OS type Linux.	Raspberry Pi pour traitement plus lourd.

8. Interfaces de communication capteur-MCU

Interface	Fils principaux	Usage
I2C	SDA + SCL	Plusieurs capteurs sur 2 fils ; BME280, BH1750, LCD I2C.
SPI	MOSI, MISO, SCLK, CS	Rapide ; carte SD, ecran, capteurs rapides.
UART	TX + RX	Serie ; GPS, GSM, debug Serial.
1-Wire	Une ligne de donnees	DS18B20.

9. Resistances, pull-up et pull-down

Role general des resistances : limiter le courant, proteger les composants, stabiliser les signaux et eviter les entrees flottantes. Valeurs frequentes pour pull-up/pull-down : 10 kohm ou 4,7 kohm.

Montage	Connexion	Etat par default	Quand bouton appuye
---------	-----------	------------------	---------------------

Pull-up	Resistance vers VCC	HIGH	LOW si bouton relie a GND.
Pull-down	Resistance vers GND	LOW	HIGH si bouton relie a VCC.
INPUT_PULLUP	Pull-up interne du MCU	HIGH	LOW quand bouton appuye.

Reponse type : les resistances pull-up/pull-down evitent les signaux flottants et les bruits. Pull-up maintient a HIGH, pull-down maintient a LOW.

Chapitre 3 - Protocoles de communication IoT

1. Definition et role

Un protocole de communication est un ensemble de regles qui definissent comment les donnees sont formatees, transmises, recues et interpretees entre dispositifs.

En IoT, le protocole doit souvent etre leger car les objets ont peu de memoire, peu de puissance de calcul, une batterie limitee et parfois une bande passante faible.

2. Couches reseau utiles en IoT

Couche	Role	Exemples
Application	Format et logique d echange.	MQTT, CoAP, HTTP, LwM2M.
Transport	Fiabilite ou rapidite.	TCP, UDP.
Reseau	Adressage et routage.	IPv6, 6LoWPAN.
Liaison	Acces au medium, MAC, erreurs.	IEEE 802.15.4.
Physique	Transmission radio.	Wi-Fi, LoRa, BLE, Zigbee.

3. Triangle des compromis : portee - debit - energie

Il est impossible d optimiser simultanement la portee, le debit et la consommation energetique. Le choix du protocole est donc un compromis.

Technologie	Portee	Debit	Energie	Usage
Wi-Fi	Moyenne	Eleve	Elevee	Camera, maison, donnees lourdes.
BLE	Courte	Moyen/faible	Faible	Bracelet, capteur proche smartphone.
Zigbee	Courte/moyenne	Faible	Faible	Domotique mesh.
LoRaWAN	Tres longue	Tres faible	Tres faible	Agriculture, smart city, capteurs ruraux.
NB-IoT	Longue	Faible/moyen	Faible	Compteurs intelligents, smart city.

4. Protocoles reseau sans fil

Protocole	Avantages	Limites	Choix typique
Wi-Fi	Haut debit, facile, compatible routeurs.	Consommation elevee, portee limitee, pas ideal batterie.	Donnees volumineuses, camera, alimentation secteur.
BLE	Tres faible consommation, smartphone compatible.	Courte portee, pas pour 5 km.	Objets personnels, beacons.
Zigbee	Faible energie, reseau mesh.	Debit faible, besoin passerelle.	Domotique, capteurs maison.
LoRaWAN	Longue portee, faible energie.	Tres faible debit, pas video.	Capteurs eloignes, agriculture, lampadaires.
NB-IoT	Longue portee, reseau	Depend operateur, debit limite.	Compteurs, smart city.

	opérateur, bonne couverture indoor.		
--	-------------------------------------	--	--

5. MQTT

MQTT signifie Message Queuing Telemetry Transport. C est un protocole léger de couche application, conçu pour l'IoT et le M2M. Il utilise TCP et le modèle publish/subscribe.

Element MQTT	Role
Publisher	Client qui publie un message sur un topic.
Subscriber	Client abonné à un topic pour recevoir les messages.
Broker	Serveur central qui reçoit, filtre et redistribue les messages.
Topic	Sujet logique du message, ex : maison/salon/temp.

QoS MQTT

QoS	Nom	Garantie	Usage
0	At most once	Au plus une fois, sans confirmation.	Données non critiques : température régulière.
1	At least once	Au moins une fois, duplicats possibles.	Alertes importantes : intrusion, fumée.
2	Exactly once	Exactement une fois, pas de doublon.	Commandes critiques : arrêt machine, verrouillage.

6. CoAP et HTTP

Critère	MQTT	CoAP	HTTP
Modèle	Publish/subscribe	Request/response	Request/response
Transport	TCP	UDP	TCP
Overhead	Faible	Tres faible	Élevé
Intermédiaire	Broker	Pas obligatoire	Serveur web/API
Usage privilégié	Telemetry IoT, plusieurs abonnés.	Objets très contraints.	API web/cloud, intégration simple.

CoAP est proche de HTTP mais plus léger et basé sur UDP. HTTP est simple et universel, mais plus lourd pour des objets contraints.

7. Modèles D2D et D2C

Modèle	Définition	Exemple	Avantage
D2D	Device-to-Device : communication directe entre objets.	PIR allume directement une ampoule.	Latence minimale, fonctionnement local.
D2C	Device-to-Cloud : objet vers plateforme cloud.	Thermostat vers application cloud.	Historique, analyse, contrôle à distance.

8. Choix rapide selon scénario

Scénario	Protocole conseillé	Pourquoi
Capteur agricole à 5 km	LoRaWAN ou NB-IoT	Longue portée, faible énergie.
Caméra HD	Wi-Fi ou 4G/5G	Haut débit nécessaire.
Bracelet connecté	BLE	Faible énergie et courte distance.

Maison intelligente mesh	Zigbee	Reseau maille basse consommation.
Temperature vers dashboard	MQTT	Leger, publish/subscribe, broker.

Chapitre 4 - Pre-stockage et traitement des donnees IoT

1. Pourquoi pretraiter ?

Les donnees IoT sont souvent brutees, incompletes, volumineuses, heterogenes et parfois incoherentes. Le pretraitement augmente la fiabilite, reduit la bande passante, diminue les couts de stockage et ameliore l'analyse/IA.

2. Caracteristiques 3V

V	Signification	Exemple IoT
Volume	Beaucoup de donnees generees continuellement.	Des milliers de capteurs.
Velocite	Donnees rapides, parfois temps reel.	Mesure chaque seconde.
Variete	Formats et sources differents.	JSON, CSV, images, logs, series temporelles.

3. Cycle de vie avant stockage

Etape	Role	Exemple
Acquisition	Captation et conversion numerique.	DHT11 lit 25 deg C.
Filtrage local	Eliminer erreurs evidentes au niveau Edge.	Ignorer temperature impossible.
Nettoyage	Outliers, valeurs manquantes, normalisation.	Remplacer valeur manquante.
Aggregation	Reduire le volume.	Moyenne/min/max par fenetre temporelle.

4. Filtrage et danger du filtrage excessif

Le filtrage supprime ou corrige les valeurs inutiles, brutees ou incoherentes. Mais un filtrage excessif peut supprimer une vraie anomalie critique : hausse brutale de temperature = incendie possible, fuite de gaz, vol de carburant, surchauffe machine.

Reponse type : il faut distinguer une erreur de mesure d'un evenement reel. Supprimer un Event Outlier peut faire perdre l'information la plus importante.

5. Outliers

Type	Definition	Action
Noise Outlier	Valeur aberrante due au bruit, capteur defectueux, parasite ou bug.	Corriger ou supprimer.
Event Outlier	Anomalie reelle correspondant a un evenement important.	Conservier et signaler.

Methodes de detection

Methode	Principe	A retenir
Boxplot	Quartiles et moustaches.	Points hors moustaches = outliers.
Nuage de points	Visualise les relations entre variables.	Points eloignes du groupe principal.
Histogramme	Distribution des valeurs.	Barres isolees loin du reste.
IQR	$IQR = Q3 - Q1$.	Outlier si $< Q1 - 1.5 \cdot IQR$ ou $> Q3 + 1.5 \cdot IQR$.

		1.5*IQR.
Z-score	$Z = (x - \text{moyenne}) / \text{ecart-type.}$	Outlier souvent si $ Z > 3$.
MAD	Base sur mediane.	Plus robuste si beaucoup de pics.

6. Valeurs manquantes, normalisation, aggregation

Technique	But	Exemple
Valeurs manquantes	Gerer une mesure absente.	Supprimer, moyenne, derniere valeur, interpolation.
Normalisation	Mettre memes unites/formats/echelles.	Kelvin -> Celsius ; date ISO.
Aggregation	Reduire volume par statistiques.	Moyenne, min, max, somme par minute.
Moyenne glissante	Lisser le signal et reduire le bruit.	Moyenne des N dernieres mesures.

Question probable : pourquoi utiliser des moyennes glissantes ? Reponse : pour reduire le bruit des capteurs et stabiliser les mesures. Attention : une fenetre trop grande peut masquer une alerte.

7. Edge, Fog, Cloud - details operationnels

Critere	Edge	Fog	Cloud
Position	Objet/capteur/ESP32.	Passerelle/serveur local.	Datacenter distant.
Latence	Tres faible.	Faible.	Plus elevee.
Puissance	Limitee.	Moyenne.	Tres elevee.
Usage	Decision immediate, filtrage local.	Aggregation locale distribuee.	Historique, IA, BI, stockage massif.
Exemple	ESP32 active relais si $T > 40$.	Gateway agrege capteurs usine.	Cloud analyse tendances annuelles.

8. Bases de donnees adaptees a l IoT

Type	Exemples	Utilisation IoT
Relationnelle	MySQL, PostgreSQL	Utilisateurs, equipements, zones, configurations, donnees metier structurees.
NoSQL documents	MongoDB	JSON capteurs, donnees semi-structurees, schema flexible.
NoSQL distribuee	Cassandra	Tres gros volumes, haute disponibilite.
Series temporelles	InfluxDB, TimescaleDB	Mesures horodatees : temperature, humidite, energie.
Data Lake	S3, Hadoop	Donnees brutes massives, logs, fichiers.

InfluxDB

InfluxDB est optimisee pour les series temporelles. Concepts : measurement = nom de la mesure, tags = metadonnees indexees, fields = valeurs non indexees, timestamp = temps de la mesure, Flux = langage de requete.

Architecture polyglotte / hybride

Une architecture polyglotte utilise plusieurs bases selon la nature des donnees : InfluxDB pour les mesures temporelles, PostgreSQL/MySQL pour les donnees metier, MongoDB pour les JSON, Cassandra pour les gros volumes. Une seule base n est pas optimale pour toutes les donnees IoT.

Chapitre 5 - Integration des donnees IoT

1. Objectif

L integration construit la chaine complete qui recoit les donnees des objets, les transforme, les stocke, les visualise et permet la prise de decision.

Capteur -> ESP32 -> MQTT/HTTP -> Broker/API -> ETL -> Base de donnees -> Dashboard/IA/Alerte

2. ETL : Extraction, Transformation, Chargement

Etape	Role	Exemple IoT
Extraction	Collecter les donnees brutes sans les modifier.	MQTT, HTTP/REST, CoAP, CSV, JSON, gateway.
Transformation	Nettoyer, filtrer, normaliser, enrichir, agreger.	temp=22.5 -> JSON normalise avec device_id et zone.
Chargement	Stocker dans un systeme cible.	InfluxDB, MongoDB, PostgreSQL, Data Lake, Cloud.

Reponse courte examen : ETL = Extraction, Transformation, Chargement.

3. PUSH vs PULL

Mode	Principe	Avantage	Limite
PUSH	L objet envoie les donnees des qu elles sont disponibles.	Temps reel.	Besoin infrastructure reactive.
PULL	Le systeme demande periodiquement les donnees.	Controle du flux.	Pas vraiment temps reel.

4. ETL classique vs ETL IoT

Critere	ETL classique	ETL IoT
Sources	ERP, CRM, SQL.	Capteurs, gateways, MQTT, CoAP.
Frequence	Batch quotidien/hebdo.	Streaming temps reel.
Format	Structure.	JSON, binaire, series temporelles, logs.
Volume	Previsible.	Massif et imprevisible.
Stockage	Data Warehouse.	Data Lake, NoSQL, time-series.

5. Batch vs Streaming

Critere	Batch	Streaming
Principe	Traite des donnees accumulees.	Traite les donnees en continu.
Latence	Elevee : minutes, heures, jours.	Faible : ms a secondes.
Usage	Rapports historiques, consommation mensuelle.	Alertes, anomalies, reaction immediate.
Outils	Hadoop MapReduce, Spark Batch, Talend.	Kafka, Spark Streaming, NiFi.

6. Outils d integration

Outil	Role	Pourquoi en IoT
Apache NiFi	Collecter, router, transformer, superviser les flux.	Pipeline visuel entre MQTT/API/bases.
Minifi	Version legere de NiFi pour	Collecte locale sur ressources limitees.

	Edge/gateway.	
Kafka	Plateforme de streaming distribuee.	Flux massifs, decouplage producteurs/consommateurs.
Spark Streaming	Traitement temps reel/micro-batch.	Anomalies, statistiques, fenetres temporelles.

Kafka est souvent utilise car il gere des flux massifs, le streaming et le decouplage des systemes.

7. Cloud IoT

Les services cloud IoT permettent de connecter les objets, recevoir les donnees, les stocker, les analyser, visualiser les resultats, declencher des alertes et gerer les appareils a distance. Exemples : AWS IoT, Azure IoT, Google Cloud IoT.

8. Architecture integree type smart city

Niveau	Elements
Terrain	LDR, PIR, capteur courant/tension, LED/relais.
Edge	ESP32 lit, filtre, calcule moyenne glissante, agit localement.
Communication	LoRaWAN/NB-IoT pour longue portee, MQTT pour publication.
Ingestion	Broker MQTT, NiFi ou Kafka.
Transformation	Nettoyage, normalisation, ajout device_id/zone, aggregation.
Stockage	InfluxDB mesures, PostgreSQL metier, MongoDB messages JSON.
Application	Grafana/dashboard, alertes de maintenance.
Metier	Economie d energie, securite publique, maintenance predictive.

Partie code Arduino / ESP32

1. Structure de base

```
void setup() {
  // Execute une seule fois au demarrage : Serial, pinMode, Wi-Fi, MQTT
}

void loop() {
  // Execute en boucle : lire capteurs, tester conditions, agir, envoyer donnees
}
```

setup initialise le systeme. loop contient le comportement principal et tourne continuellement.

2. Fonctions essentielles

Fonction	Role	Exemple
pinMode(PIN, OUTPUT)	Configurer une broche en sortie.	LED, relais, buzzer.
pinMode(PIN, INPUT)	Configurer une broche en entree.	Bouton, PIR.
pinMode(PIN, INPUT_PULLUP)	Entree avec pull-up interne.	Bouton : HIGH par defaut, LOW appuye.
digitalRead(PIN)	Lire HIGH/LOW.	Etat bouton.
digitalWrite(PIN, HIGH/LOW)	Commander une sortie.	Allumer LED, activer relais.
analogRead(PIN)	Lire valeur analogique.	LDR, potentiometre.

3. delay() vs millis()

Critere	delay()	millis()
Fonctionnement	Bloque le programme.	Non bloquant.

Pendant attente	MCU ne lit plus capteurs/boutons.	MCU continue les autres taches.
IoT reactif	Mauvais choix.	Meilleur choix.
Exemple	delay(5000) bloque 5 s.	Verifier si 5 s sont passees.

Reponse type : delay bloque l execution ; millis permet une temporisation non bloquante, donc le systeme reste reactif aux capteurs, boutons et communications.

4. INPUT_PULLUP

```
pinMode(BUTTON_PIN, INPUT_PULLUP);

if (digitalRead(BUTTON_PIN) == LOW) {
    digitalWrite(LED_PIN, HIGH);
}
```

Avec INPUT_PULLUP, l'entree est HIGH par default. Quand le bouton est presse, il relie la broche a GND, donc la lecture devient LOW. Ce code allume la LED quand le bouton est presse.

5. Code relais et temperature

```
if (temperature > 40) {
    digitalWrite(RELAY, HIGH);
}
```

Explication type : le microcontrôleur verifie si la temperature depasse 40 deg C. Si oui, il active le relais. Le relais peut declencher un ventilateur, une alarme ou un systeme de refroidissement. C est une decision locale Edge.

6. Probleme d envoi au cloud sans temporisation

```
void loop(){
    float temperature = readTemperature();
    sendDataToCloud(temperature);
}
```

Probleme : le code envoie les donnees a chaque tour de boucle, sans delai, filtrage ou condition. Cela peut saturer le reseau, augmenter la consommation, surcharger le serveur et stocker trop de donnees inutiles.

Amelioration : envoyer toutes les N secondes avec millis(), envoyer seulement si variation significative, ou agreger localement.

7. Relais 5 secondes avec millis()

```
#define BUTTON 4
#define RELAY 5

bool relayActive = false;
unsigned long startTime = 0;
const unsigned long duration = 5000;

void setup() {
    pinMode(BUTTON, INPUT_PULLUP);
    pinMode(RELAY, OUTPUT);
    digitalWrite(RELAY, LOW);
}

void loop() {
    if (digitalRead(BUTTON) == LOW && relayActive == false) {
        digitalWrite(RELAY, HIGH);
        relayActive = true;
        startTime = millis();
    }

    if (relayActive == true && millis() - startTime >= duration) {
        digitalWrite(RELAY, LOW);
    }
}
```

```

    relayActive = false;
  }
}

```

Ce code active le relais pendant 5 secondes sans bloquer le programme. Pendant ce temps, le microcontrôleur peut continuer à lire d'autres capteurs.

8. LDR + LED

```

int luminosite = analogRead(LDR_PIN);

if (luminosite < seuil) {
  digitalWrite(LED_PIN, HIGH); // sombre : allumer
} else {
  digitalWrite(LED_PIN, LOW);   // clair : éteindre
}

```

La logique peut être inversée selon le montage LDR. Toujours lire l'énoncé : parfois la valeur augmente avec la lumière, parfois elle diminue.

9. Pipeline logique dans un code IoT

1. Lire capteur
2. Vérifier validité
3. Filtrer / agréger
4. Décider selon seuil
5. Commander actionneur
6. Envoyer au cloud

Questions types examen et réponses courtes

Question	Réponse attendue
Definition IoT	Interconnexion d'objets physiques via Internet pour collecter, traiter, transmettre des données et agir.
4 composants objet connecté	Capteurs/actionneurs, processeur, module communication, énergie.
3 couches IoT	Perception, réseau, application.
5 couches IoT	Perception, transport/réseau, traitement, application, métier.
M2M vs IoT	M2M = machine à machine ; IoT ajoute cloud, applications, analyse et métier.
Capteur vs actionneur	Capteur mesure ; actionneur agit.
Analogique/numérique/TOR	Continu / code discret / 0 ou 1.
Transducteur	Convertit une grandeur physique en signal électrique.
DS18B20	Température numérique, protocole 1-Wire, -55 à +125 °C.
PIR	Détecte variations infrarouges des corps en mouvement.
MQTT	Protocole léger publish/subscribe sur TCP avec broker.
QoS MQTT	0 au plus une fois ; 1 au moins une fois ; 2 exactement une fois.
CoAP	Protocole léger request/response sur UDP.
Triangle compromis	Impossible d'optimiser portée, débit et énergie en même temps.
Wi-Fi pas toujours adapté	Consommation élevée et portée limitée pour capteurs batterie.
BLE à 5 km	Erreur : BLE courte portée ; utiliser LoRaWAN/NB-IoT.
Prétraitement	Filtrer, nettoyer, normaliser, agréger pour rendre données exploitables.
Noise vs Event Outlier	Erreur/bruit à supprimer vs vraie anomalie à signaler.
Moyenne glissante	Lisser les mesures et réduire le bruit.
Edge/Fog/Cloud	Objet proche / passerelle intermédiaire / datacenter distant.
Architecture polyglotte	Plusieurs bases selon données : InfluxDB, PostgreSQL,

	MongoDB.
ETL	Extraction, Transformation, Chargement.
Kafka en IoT	Flux massifs, streaming, decouplage des systemes.
MiNiFi	NiFi leger pour passerelles Edge.
delay vs millis	delay bloque ; millis ne bloque pas.
INPUT_PULLUP	HIGH par default, LOW quand bouton presse.

Methode pour repondre a une etude de cas IoT

Quand on te donne un cas, reponds toujours avec cette structure pour ne rien oublier :

1. Identifier le besoin : surveillance, controle, alerte, economie, securite.
2. Identifier les capteurs et actionneurs.
3. Choisir le microcontroleur ou passerelle.
4. Choisir le protocole selon portee, debit, energie et latence.
5. Preciser Edge/Fog/Cloud et le pretraitement local.
6. Construire le pipeline : acquisition -> communication -> ETL -> stockage -> dashboard.
7. Choisir les bases : time-series, relationnelle, NoSQL selon les donnees.
8. Ajouter alertes, securite et limites/ameliorations.

Exemple final : eclaireage public intelligent

Partie	Reponse
Besoin	Reduire consommation et detecter pannes.
Capteurs	LDR luminosite, PIR presence, courant/tension consommation.
Actionneurs	LED, variateur, relais/driver.
MCU / Edge	ESP32 ou controleur par lampadaire, filtrage et decision locale.
Communication	LoRaWAN ou NB-IoT pour longue portee ; MQTT pour messages.
Traitement	Moyenne glissante, filtrage outliers, detection panne.
Stockage	InfluxDB mesures, PostgreSQL equipements/zones, MongoDB logs JSON.
Application	Dashboard Grafana, alertes maintenance, reporting energie.

Derniere page - ultra resume

A apprendre par coeur :

- IoT = capter + communiquer + traiter + agir.
- Objet IoT = capteurs/actionneurs + processeur + communication + energie.
- 3 couches = perception, reseau, application. 5 couches = perception, transport, traitement, application, metier.
- Capteur mesure ; actionneur agit ; transducteur convertit physique en electrique.
- Analogique = continu ; numerique = code ; TOR = 0/1.
- Pull-up = HIGH par default ; pull-down = LOW par default ; INPUT_PULLUP = LOW quand bouton appuye.
- MQTT = publish/subscribe + broker + TCP. CoAP = request/response + UDP. HTTP = plus lourd.
- Triangle : longue portee + faible energie = faible debit ; Wi-Fi = haut debit mais energivore ; BLE = courte portee faible energie.
- Pretraitement = filtrage + nettoyage + normalisation + aggregation.
- Ne pas supprimer tous les outliers : Noise Outlier vs Event Outlier.
- Edge = pres du capteur ; Fog = passerelle ; Cloud = datacenter.
- BDD polyglotte : InfluxDB time-series, PostgreSQL/MySQL metier, MongoDB JSON.
- ETL = Extraction, Transformation, Chargement ; Kafka = streaming massif ; NiFi/MiNiFi = pipelines.
- delay bloque ; millis ne bloque pas. Toujours expliquer ce que le code lit, decide, actionne et envoie.